

HYDRA算法优化与论文实验

4.6

在实际的运用过程中如果使用HYDRA_exact的算法，对于每一个窗口都需要计算最近邻的窗口，这时的时间复杂度可以表示为 $O(n^2m)$ ，时间序列很长、产生的窗口数量 n 很大时，计算量会迅速增加，由于 HYDRA 的层级筛选机制并不要求邻居关系完全精确，因此作者利用已有的 ANNS，并使用 HNSW 实现 HYDRA_approx，将复杂度降低到 $O(n \log n \cdot m)$ ，同时基本保持检测准确率。

4.7

在针对时间序列多变量数据的时候，计算距离的方法依然适用并且有很好的筛选效果
具体步骤如下：

- 1.先对已有数据的每一个channel的数据进行全局z-score归一化
- 2.切割窗口得到多个矩阵
- 3.两个矩阵之间距离的平方=两个矩阵每个相对应的数据之差的平方的和
- 4.两个矩阵之间的距离=3所得的数开根号

5.1

作者使用 TSB-AD 数据集，在单变量和多变量时间序列上与多种异常检测算法比较，主要用 **VUS-PR** 衡量（越大越表明预测准确）检测效果

5.2

HYDRA 在单变量和多变量实验中总体表现最好，说明这说明前面提出的层级代表筛选 + 多层异常分数聚合，相比单纯使用最近邻或者聚类，能够更加稳定地检测异常

5.3

前面论文提出的问题是：

- MP容易漏掉多个相似异常
- clustering方法又可能漏掉细小、孤立异常

因此作者进一步按照不同数据领域和异常类型进行比较

结果发现MP和NormA各自在某些特定类型异常上表现较好，而HYDRA整体更加稳定

验证了：

HYDRA的多层结构确实能够同时兼顾不同尺度、不同形式的异常，而不是只针对某一种异常。

5.4

实验比较 NoNorm、Z-score、Min-Max 和 Unit-length 后发现，整体上不进行窗口级归一化效果最好。

HYDRA通过直接使用原始窗口 + 层内异常分数标准化 + 多层结果综合，降低了算法对某一种固定窗口归一化方式的依赖

5.5

作者改变这个算法中的不同参数值，验证他们对结果的影响：

1. 窗口长度：ACF自动选择窗口长度比固定设置某一个长度整体更加稳定
2. 多层聚合：使用多个层级明显优于只使用单层，说明层级结构本身确实提供了额外信息
3. 层级深度：加入更深层级以后整体检测效果继续提高，而且越深层代表窗口越少，新增计算量反而比较小
4. Exact和Approx：使用HNSW近似最近邻后速度大幅提高，但准确率基本没有下降

5.6

4.6提出HYDRA_exact的计算量过大，因此使用HNSW得到HYDRA_approx

经过实验验证得到：时间序列越长，HYDRA_approx相对于精确最近邻的速度优势越明显，同时仍然保持较好的检测效果

5.7

局限：HYDRA仍然受到“正常模式总体占多数，并且能够形成稳定代表”这一前提的约束
因此当——正常数据本身非常复杂、差异很大或异常只是幅度很小的高频抖动时
HYDRA仍可能出现误报或者漏报